

FUNCr: Binary Function Name Recovery via Graph-Based Multi-Context Embeddings and Retrieval

Ananta Dian Pradipta
New Jersey Institute of Technology
Newark, NJ, USA
adp232@njit.edu

Zhihao Lin
New Jersey Institute of Technology
Newark, NJ, USA
zl74@njit.edu

Robert Blacha
New Jersey Institute of Technology
Newark, NJ, USA
rb236@njit.edu

Haotian Zhang
New Jersey Institute of Technology
Newark, NJ, USA
haotian.zhang@njit.edu

ABSTRACT

Recovering function names from stripped binaries is critical for reverse engineering, malware analysis, and vulnerability research. We present FUNCr, a graph neural network-based system that learns function representations by fusing multiple context sources (external library calls, callee signatures, and caller signatures) through a cascaded gated fusion mechanism with conditional bypass. Our ablation study reveals a novel *ext-call paradox*: external call information alone improves performance on seen binaries but *degrades* generalization to unseen packages by 4.3 percentage points; inter-procedural callee/caller context is required for disambiguation. Furthermore, we demonstrate that nearest-neighbor retrieval over learned encoder embeddings outperforms autoregressive sequence decoding, improving exact match by 4.7 percentage points on unseen packages with zero retraining. This finding suggests that for well-trained encoders, retrieval over embeddings outperforms autoregressive generation, where the encoder representation, not the decoder, is the key component. Evaluated on 300,013 functions from 77 packages, FUNCr achieves 0.779 F1 on a held-out test set (13,301 functions) and 0.650 F1 across four completely unseen packages (10,467 functions), outperforming a reproduced SYMGEN checkpoint on the same cross-project test set (0.450 F1) while using only 32M parameters, roughly 1000× smaller than SYMGEN’s 34B-parameter CodeLlama backbone.

KEYWORDS

binary analysis, function name recovery, graph neural networks, gated fusion, retrieval-augmented inference

1 INTRODUCTION

Software reverse engineering is a fundamental activity in cybersecurity, requiring analysts to understand the behavior of compiled binary programs without access to source code. Modern compilers strip all symbolic information (function names, variable names, type annotations) from release binaries, leaving analysts with thousands of unnamed functions (e.g., `sub_4a30`, `FUN_00105c10`) that must be manually identified. Recovering meaningful function names dramatically accelerates program comprehension: knowing that `sub_4a30` is `xmalloc` immediately conveys its purpose.

The practical importance of automated function name recovery extends across multiple domains of cybersecurity. In malware

analysis, meaningful function names reduce triage time from hours to minutes, allowing analysts to quickly identify cryptographic routines, network communication handlers, and persistence mechanisms. In vulnerability research, understanding stripped firmware is essential for discovering exploitable bugs in IoT devices and embedded systems, where vendors rarely ship debug symbols. In competitive reverse engineering (CTFs) and forensic investigations, even partial name recovery provides critical footholds for understanding program logic. Commercial tools such as IDA Pro and Ghidra provide manual annotation workflows and limited pattern-matching (FLIRT signatures), but cannot automatically recover names for arbitrary functions; they rely on exact byte-pattern matching against known library signatures, which fails for recompiled or modified code.

Prior work on automated function name recovery has explored statistical approaches [1], extreme multi-label classification [6], graph neural networks [2], language model pre-training [3], ensemble embedding with contrastive captioning [4], and large language model fine-tuning [5, 7]. However, each approach has significant limitations. DeBin [1] and XFL [6] use closed-vocabulary classifiers that cannot generate names absent from training; recent reproduction shows XFL abstains on 60.7% of cross-project functions and SymLM’s 3069-class classifier fails on 43.8%. NERO [2] achieves F1=0.455 using pointer-aware slicing and GNNs on call graphs, but only works for functions *with* external calls—a fundamental limitation since approximately 70% of functions in typical binaries have none. SymLM [3] reaches F1=0.585 on its own test set by pre-training on micro-execution traces, but requires dynamic analysis infrastructure (executing binaries in an emulator), limiting applicability to binaries that can be safely run. More recently, BLens [4] achieves 0.79 F1 per-binary but drops to 0.46 on cross-project evaluation (a 42% degradation), and SYMGEN [5] fine-tunes CodeLlama-34B but reports only 0.38 cross-project F1 after proper deduplication, demonstrating that billion-parameter models do not automatically generalize. Two fundamental challenges remain underexplored: (1) how to effectively combine multiple sources of contextual information (code structure, library calls, and inter-procedural relationships) without degrading generalization, and (2) whether autoregressive name generation is the right output paradigm, given that function names are drawn from a relatively small vocabulary of real names.

We address these challenges with FUNCNR, a graph neural network system that lifts stripped binaries to BAP intermediate representation, encodes each function’s control flow graph with a GAT-based encoder, and fuses multiple sources of inter-procedural context through a cascaded gated mechanism with conditional bypass. Self-supervised pretraining (masked language modeling and contrastive learning) with partial embedding transfer further strengthens the encoder representations. At inference time, we demonstrate that simple k -nearest-neighbor retrieval over the learned embeddings outperforms autoregressive beam search decoding, improving cross-project exact match by 4.7 percentage points with zero retraining.

Evaluated on 300,013 functions from 77 packages, FUNCNR achieves 0.779 F1 on a held-out test set (13,301 functions) and 0.650 F1 across four completely unseen packages (10,467 functions spanning `tengine`, `angie`, `nginx118`, and `recutils`). On the same cross-project test set, we reproduce the released SYMGEN checkpoint and measure 0.450 F1, yielding a +0.20 absolute improvement for FUNCNR with a model roughly 1000× smaller (32M versus 34B parameters).

In summary, we make the following contributions:

- We design FUNCNR, a graph neural network system for binary function name recovery that incorporates a novel cascaded gated fusion mechanism. This 3-stage architecture sequentially fuses external library calls, callee signatures, and caller signatures with conditional bypass, where each stage is skipped when the corresponding context is absent. Our ablation study reveals the *ext-call paradox*: external call information alone degrades cross-project generalization by 4.3 percentage points, while adding inter-procedural callee/caller context resolves this and improves cross-project EM by 12.8 percentage points.
- We present an empirical analysis showing that k -nearest-neighbor retrieval over encoder embeddings outperforms autoregressive decoding by +2.0pp test EM and +4.7pp cross-project EM with zero retraining. This finding, consistent with kNN-LM [20], demonstrates that for our encoder architecture, representation quality is the primary bottleneck rather than the decoding strategy.
- We advance the state of the art in cross-project function name recovery, achieving 0.779 F1 on a held-out test set (13,301 functions) and 0.650 F1 across four completely unseen packages (10,467 functions). On this cross-project test set, we reproduce the released SYMGEN checkpoint and measure 0.450 F1 (a +0.20 absolute gap) despite using a model 1000× smaller than SYMGEN’s 34B-parameter backbone. Additionally, self-supervised pretraining with partial embedding transfer provides +15.9 percentage point cross-project EM improvement. Our code and dataset are publicly available at <https://github.com/ananta-pradipta/stripped-binary-name-recovery>.

2 BACKGROUND AND MOTIVATION

2.1 Problem Definition

We formalize binary function name recovery as follows. Given a stripped binary B containing n functions $\{f_1, f_2, \dots, f_n\}$, where each function f_i is represented by its control flow graph $G_i = (V_i, E_i)$ with basic blocks as nodes and control flow edges, the goal is to

predict a human-readable name $\hat{n}_i$ for each function that matches the developer-assigned name n_i from the original source code.

Each function name is decomposed into a sequence of sub-tokens using a vocabulary $\mathcal{V}$ of 7,004 tokens: $n_i = (t_1, t_2, \dots, t_L)$ where $t_j \in \mathcal{V}$ and $L \leq 20$. For example, `quotearg_buffer_restyled` becomes the sub-token sequence [quote, arg, _, buffer, _, re, styled]. The prediction task is thus a sequence generation problem conditioned on the function’s binary representation.

In addition to the CFG, each function may have three forms of contextual information: (1) external library calls $\mathcal{E}_i$, the set of PLT/IAT function names called by f_i (e.g., `malloc`, `fprintf`); (2) callee signatures C_i^{ee} , instruction-type token sequences of internal functions called by f_i ; and (3) caller signatures C_i^{er} , instruction-type token sequences of functions that call f_i . These contextual signals are optional: approximately 70% of functions have no external calls, and leaf functions have no callees.

2.1.1 Scope and Assumptions. We scope FUNCNR to the following setting:

- **Architecture:** x86-64 ELF binaries compiled for Linux.
- **Optimization levels:** O0 (no optimization) and O2 (standard optimization). We do not evaluate O1 or O3, as O1 is rarely used in practice and O3 introduces aggressive inlining that duplicates function bodies.
- **Compilation:** GCC compiler. We do not evaluate Clang or MSVC, though the instruction-type tokenization is compiler-agnostic at the IR level.
- **Training corpus:** We assume access to a corpus of labeled binaries where ground-truth function names are known (obtained from debug builds). This is a standard assumption in the literature [1–5].
- **No obfuscation:** We assume binaries are not deliberately obfuscated. Obfuscation techniques (control flow flattening, opaque predicates, symbol stripping with anti-disassembly) would require additional preprocessing.

2.1.2 Evaluation Scenario. We evaluate in two settings: (1) a held-out test set of 13,301 functions from binaries of packages seen during training (cross-binary evaluation), and (2) a cross-project set of 10,467 functions from four packages (`tengine`, `angie`, `nginx118`, `recutils`) completely excluded from training. The cross-project setting is stricter than the cross-binary evaluation used by some prior work [2, 3], where different binaries of the *same* package may appear in both training and test. Cross-project evaluation tests whether the model generalizes to entirely new codebases with potentially novel naming conventions, and is the evaluation methodology advocated by SYMGEN [5].

2.2 Challenges

Binary function name recovery presents several fundamental challenges that motivate our design decisions:

Token collision. After instruction-type tokenization, many structurally different functions share similar token distributions. Two unrelated functions may both consist primarily of `MEM_READ_64`, `ARITH_ADD`, and `STACK_STORE_64` tokens, making them difficult to distinguish from instruction tokens alone. This motivates our multi-context fusion: inter-procedural context (what a function calls and

who calls it) provides disambiguating signal that code structure alone cannot.

Mode collapse. When contextual features are naively incorporated, the fusion mechanism must produce meaningful output even when no context exists (e.g., a function with no external calls receives a zero-vector context input). In practice, this causes the gate to learn a default output, and 47% of all predictions collapse to a single name (`xmalloc`, the most frequent function). Our conditional bypass mechanism addresses this by skipping fusion stages entirely when context is absent.

Cross-project generalization gap. Models that perform well on held-out test sets from seen packages often degrade dramatically on entirely unseen packages. BLens [4] drops from 0.79 to 0.46 F1 (42% degradation), and our own ablation reveals that external call information—which helps on seen binaries—actually *hurts* cross-project generalization by 4.3 percentage points (the ext-call paradox). This motivates careful fusion design with conditional bypass and inter-procedural disambiguation.

O0/O2 optimization gap. Binaries compiled without optimization (O0) produce many small wrapper functions with similar boilerplate stack frame setup/teardown, where two unrelated functions may have 70% token overlap from stack operations alone. O2-compiled code eliminates this boilerplate through register allocation and inlining, leaving more distinctive patterns. This gap (41.5% vs. 69.8% EM on our test set) motivates our contrastive pretraining objective that learns optimization-invariant representations.

2.3 Related Work and Our Motivations

2.3.1 Binary Function Name Recovery. He et al. [1] introduced DeBin, which uses BAP intermediate representation with probabilistic graphical models (ExtraTrees + CRF) to predict debug information including function names. While pioneering, DeBin’s statistical approach cannot capture the complex patterns that deep learning exploits. We use DeBin as the non-neural baseline in our ablation (Table 2).

David et al. [2] proposed NERO, using pointer-aware slicing and GNNs on call graphs to predict function names, achieving F1=0.455. NERO’s key insight is that call-site context matters for naming. However, NERO only works for functions *with* external calls, a fundamental limitation since 70% of functions in typical binaries have none. Our conditional bypass mechanism (Section 4.4) addresses this directly: functions without calls use only code features, while functions with calls benefit from multi-context fusion.

Jin et al. [3] presented SymLM, which pre-trains on micro-execution traces and fuses caller/callee context for function name prediction (F1=0.585). SymLM’s “fusing encoder” concatenates context embeddings, whereas our gated fusion learns a per-function balance between code and context features with conditional bypass. More critically, SymLM requires dynamic analysis infrastructure (executing binaries in an emulator), whereas FUNCR operates purely on static analysis. Our ablation reveals the ext-call paradox (Section 5.2), a phenomenon SymLM does not analyze, showing that context fusion must be carefully designed to avoid degrading generalization.

Patrick-Evans et al. [6] introduced XFL, framing function name prediction as extreme multi-label classification over a closed tag

vocabulary, achieving 82.5% tag precision on Debian binaries. XFL’s lasting contribution is the DEXTER embedding, which subsequent ensemble systems (including BLens) reuse as a feature source. Under Benoit et al.’s [4] unified reproduction, XFL produces empty predictions for 60.7% of cross-project functions (an abstention driven by its confidence-threshold design), yielding an aggregate sub-token F1 of 0.22 on the Punstrip cross-project setting. XFL’s multi-label formulation thus operates over a closed label space by construction; FUNCR instead emits sub-token sequences and retrieves from a training-name index of approximately 276K embedding-name pairs, a larger recognition base although still bounded by training-set overlap (Section 7).

Al-Kaswan et al. [4] introduced BLens, combining ensemble embeddings (PalmTree, CLAP, DEXTER) with contrastive captioning and a specialized LORD decoder. While achieving 0.79 F1 per-binary, cross-project F1 drops to 0.46, a 42% degradation that highlights the generalization challenge we address. BLens also reproduces earlier systems under a unified evaluation: notably, SymLM cross-project F1 drops to 0.277 with 43.8% of test functions falling outside its trainable vocabulary, providing direct evidence that sub-token generation (our approach) structurally outperforms closed-class prediction on unseen packages. Xu et al. [5] fine-tuned CodeLlama-34B with LoRA for function naming, importantly demonstrating that prior results were inflated 1.2–2.9× by duplicate functions (cross-project F1=0.38 after proper deduplication). We follow SYMGEN’s deduplication methodology in our evaluation and additionally reproduce the released SYMGEN checkpoint directly on our test set for head-to-head comparison.

Jiang et al. [10] applied domain-adapted LLMs to function name inference, representing the trend toward large-scale language model approaches. Su et al. [7] proposed ProRec (NeurIPS 2024), a probe-and-recover framework that uses source code foundation models to synthesize symbol-rich context fragments fed to black-box LLMs, improving function-name token recall by 7.4 percentage points over decompilation-only prompting. GenNm [11] fine-tuned CodeGemma with SymPO preference optimization for variable name recovery, a related task demonstrating that preference-aligned generation improves binary symbol prediction. Our approach differs fundamentally from all three: rather than relying on source-code LLMs (which benefit from contamination with training-set projects) or scaling model size to billions of parameters, we invest in the *encoder representation* (32M parameters) and show that simple k -NN retrieval over these embeddings outperforms autoregressive generation, consistent with Khandelwal et al.’s finding that retrieval can outperform generation when representations are sufficiently good [20, 21].

Most closely related to our name tokenization is Epitome [8], a concurrent FSE 2024 work that uses votes-based sub-token construction for function-name prediction combined with multi-task learning. Our votes-style tokenization is developed independently on BAP intermediate representation (versus Epitome’s assembly-level LM), and we integrate it with a cascaded gated fusion of inter-procedural context rather than multi-task auxiliary losses. As Epitome’s implementation is not publicly released, we are unable to reproduce it on our test set and cite its published numbers for reference.

2.3.2 Binary Code Representation Learning. Learning effective representations of binary code underpins all downstream binary analysis tasks. Xu et al. [12] pioneered GNN-based binary analysis with Gemini, using graph embedding networks for cross-platform binary similarity detection. We adopt their insight that CFG structure is informative, extending it with GAT attention and multi-context fusion. Ding et al. [13] proposed Asm2Vec, learning function embeddings via PV-DM random walks over assembly instructions. Unlike Asm2Vec’s raw instruction tokens, our V3 tokenization abstracts instructions into 1,510 semantic types, reducing vocabulary by 95% while preserving discriminative information.

Wang et al. [14] introduced jTrans, a jump-aware Transformer that incorporates control flow structure into positional encoding for binary similarity. Pei et al. [15] developed Trex, learning execution semantics from micro-traces. Both demonstrate the value of pre-training on binary code. We adopt this principle in our MLM + contrastive pre-training stage (Section 4.5), but operate on static BAP-IR rather than execution traces.

Zhu et al. [16] proposed CALLEE for recovering call graphs via contrastive learning with transfer learning, showing +28% F1 from pre-training on related tasks. Two of their findings directly support our design: (1) transfer learning from pre-training provides substantial gains, validating our encoder pre-training approach; (2) “loose symbolization” (semantic type abstraction) outperforms strict tokenization, supporting our instruction-type tokenization.

2.3.3 GNNs for Binary Analysis. Graph neural networks are a natural fit for binary analysis, where control flow graphs (CFGs) provide structural information. Zhu et al. [17] applied GNNs to type inference on stripped binaries (TYGR), demonstrating that graph structure captures semantic properties beyond sequential instruction patterns. He et al. [18] proposed semantics-oriented graph representations for binary code similarity, arguing that “code is not natural language” and that graph-based approaches better capture program semantics than sequential models.

Our work extends GNN-based binary analysis in two ways: (1) we use graph attention networks (GAT) with learned attention over CFG edges, allowing the model to weight structurally important blocks; (2) we introduce a multi-context fusion mechanism that incorporates inter-procedural information (callee/caller signatures and library calls) not present in the CFG alone.

2.3.4 Retrieval-Augmented Generation. Our finding that k -NN retrieval outperforms autoregressive decoding connects to a growing body of work on retrieval-augmented methods in NLP. Khandelwal et al. [20] demonstrated that augmenting a neural language model with a nearest-neighbor search over cached hidden states (k NN-LM) significantly improves perplexity without any additional training. They extended this to machine translation (k NN-MT) [21], showing that retrieval-augmented translation outperforms the base model, particularly on domain-specific data. Borgeaud et al. [22] proposed RETRO, which retrieves relevant text chunks from a large database during generation, achieving performance comparable to models 25× larger.

These works share a common insight with our findings: when the output space is drawn from a fixed set of real examples (function names in training, sentences in a corpus), retrieval over learned

representations can outperform unconstrained generation. However, our setting differs in an important way: in k NN-LM/ k NN-MT, retrieval *augments* generation at the token level, whereas in our system, retrieval *replaces* generation entirely: we retrieve a complete function name rather than interpolating token-level distributions. This is because function names are atomic labels (each name is a single concept), not compositional sequences where token-level retrieval would help.

3 OVERVIEW

FuncR follows a 3-stage encoder-decoder pipeline, illustrated in Figure 1. The pipeline transforms a stripped binary into predicted function names through three phases:

Stage 1: Block Encoding. Each function’s basic blocks are tokenized into instruction-type tokens (Section 4.1.1), which are processed by a Transformer encoder to produce per-block embeddings $\mathbf{b}_i \in \mathbb{R}^{512}$.

Stage 2: Graph Encoding and Multi-Context Fusion. A Graph Attention Network (GAT) aggregates block embeddings over the control flow graph to produce a function-level embedding $\mathbf{f} \in \mathbb{R}^{1024}$. The cascaded gated fusion mechanism then sequentially incorporates external library calls, callee signatures, and caller signatures through learned gates with conditional bypass, producing the final fused embedding $\mathbf{z} \in \mathbb{R}^{1024}$.

Stage 3: Name Prediction. The fused embedding $\mathbf{z}$ feeds either a GRU decoder with beam search or k -NN retrieval over the training set embeddings, producing a predicted function name as a sequence of sub-tokens.

Prior to supervised training, the block encoder and graph encoder are jointly pre-trained with two self-supervised objectives: masked language modeling (predicting randomly masked instruction tokens) and contrastive learning (pulling together embeddings of the same function compiled at O0 and O2). This pre-training, followed by partial embedding transfer (84.4% of token embeddings matched by name), provides the encoder with a strong initialization that improves cross-project EM by 15.9 percentage points.

Figure 2 shows the detailed model architecture. The key design decision is the cascaded gated fusion with conditional bypass (Figure 3): each fusion stage is skipped entirely when the corresponding context is absent, preventing mode collapse.

4 DETAILED DESIGN

4.1 Preprocessing

We compile source packages at optimization levels O0 and O2, producing both debug and stripped binaries. We use BAP 2.5.0 [19] to lift stripped binaries to BAP Intermediate Representation (BAP-IR), which provides a normalized, architecture-independent view of the binary code.

4.1.1 Instruction-Type Tokenization. Raw BAP-IR contains over 32,000 unique tokens, many of which are register names, immediate values, or memory addresses that vary across binaries. We classify each BAP-IR instruction into one of approximately 1,510 semantic types:

- **Calls:** CALL_malloc, CALL_printf, CALL_INTERNAL

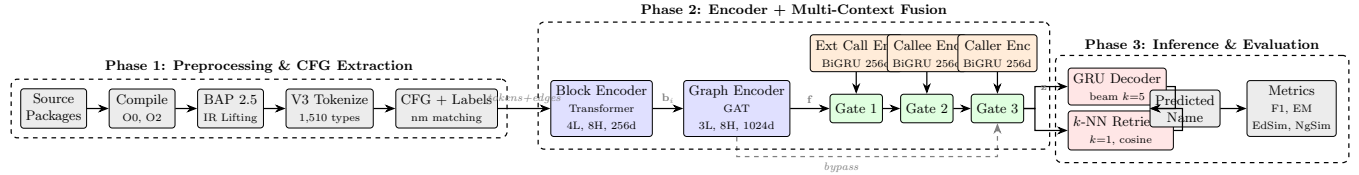


Figure 1: End-to-end FUNCR pipeline. Phase 1 compiles source packages, lifts to BAP-IR, tokenizes instructions into 1,510 semantic types, and extracts CFGs with ground-truth labels. Phase 2 encodes basic blocks (Transformer), aggregates over the CFG (GAT), and fuses library call context, callee signatures, and caller signatures through cascaded gated fusion with conditional bypass. Phase 3 produces function names via either GRU beam search or k -NN retrieval, evaluated with sub-token F1, exact match, edit similarity, and n -gram similarity.

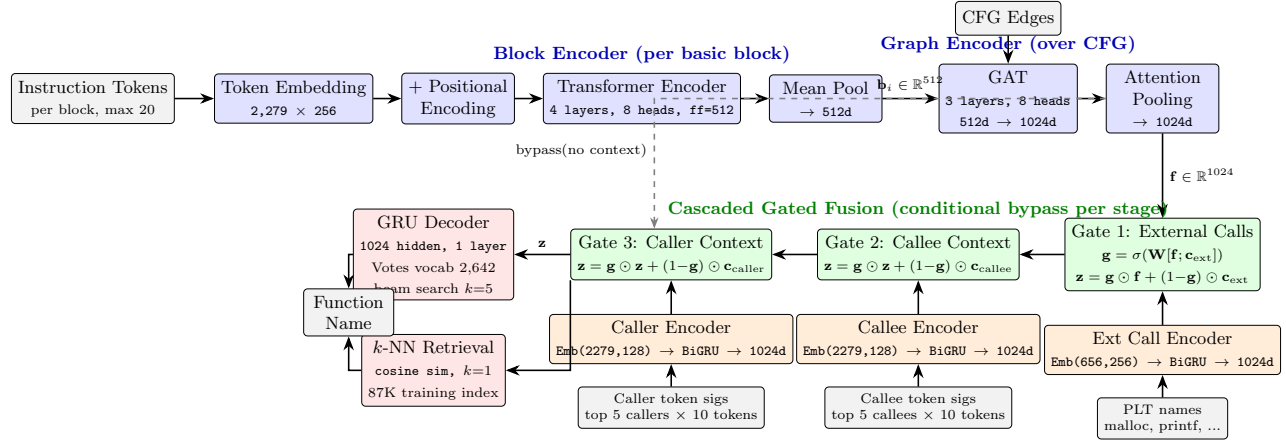


Figure 2: Detailed FUNCR model architecture. The block encoder processes instruction-type tokens per basic block via a Transformer. The graph encoder aggregates block embeddings over the CFG using GAT with attention pooling. The cascaded gated fusion sequentially incorporates external library calls, callee signatures, and caller signatures, where each stage is bypassed when the corresponding context is absent. The fused embedding z feeds either a GRU decoder (beam search) or k -NN retrieval.

- **Memory:** MEM_READ_32, MEM_WRITE_ARG_64, MEM_READ_GLOBAL_32
- **Flags:** FLAG_CF, COND_BRANCH_ZF
- **Constants:** ASSIGN_ZERO, ASSIGN_POW2, ASSIGN_ADDR (immediate value bucketing)
- **Control:** BRANCH, RETURN, COMPARE

This tokenization reduces vocabulary by 95% while preserving semantic distinctions. Our ablation shows a +1,750% F1 gain over raw BAP-IR tokens. The tokenization is deterministic and compiler-agnostic: it depends only on BAP-IR semantics, not on specific register assignments or memory layouts.

4.1.2 ENDBR64 Wrapper Resolution. In O0-compiled binaries, BAP lifts `endbr64; jmp target` sequences as separate 2-block functions. These are Intel CET (Control-flow Enforcement Technology) stubs, not real functions. We found that 96.4% of O0 functions were such wrappers, each containing only a single `CALL_INTERNAL` token. Our preprocessing detects these wrappers and replaces their graphs with the target function’s graph, raising O0 exact match from 1.6% to 41.5%.

4.1.3 Votes Name Tokenizer. Function names must be decomposed into sub-tokens for sequence prediction. Standard BPE tokenization produces poor results because function names follow naming conventions (snake_case, camelCase) that differ from natural language. We developed Votes, a 3-model voting tokenizer that achieves 95% lower out-of-vocabulary rate compared to BPE.

Votes constructs its vocabulary by corpus-frequency voting over sub-token candidates produced by a rule-based splitter (on underscores, camelCase boundaries, and digit transitions): tokens with at least `min_count` occurrences in the training corpus are kept, while rare tokens fall back to character-level decomposition. This produces a vocabulary of 7,004 sub-tokens on our 300K-function corpus that covers the vast majority of all function name tokens, compared to roughly 94% for BPE with a comparable vocabulary size. The reduced OOV rate directly improves exact match: a name cannot be correctly predicted if any of its sub-tokens are out of vocabulary.

For example, `quotearg_buffer_restyled` is tokenized as `[quote, arg, _, buffer, _, re, styled]` by Votes, whereas BPE might produce `[quo, tear, g_buf, fer_rest, yled]`, losing semantic boundaries.

4.1.4 Dataset Construction. Our dataset is constructed through a multi-step pipeline. First, we compile each source package twice: a *debug* build (with `-g` flag) and a *stripped* build (with `strip -strip-all`). The debug build provides ground-truth function names via the `nm` utility, which extracts symbol table entries mapping addresses to names. The stripped build is the input to our model and contains no symbolic information.

We lift each stripped binary using BAP 2.5.0 [19], which produces BAP-IR files containing per-function control flow graphs. BAP names functions using their entry addresses (e.g., `sub_4a30`). We match BAP function addresses to debug symbols via exact address matching, discarding functions that cannot be matched (typically compiler-inserted functions like `_start` or `__libc_csu_init`).

Data is split at the *binary* level, not the function level, to prevent data leakage. A fixed split assignment file ensures reproducibility: 137 binaries for training, 17 for validation, and 18 for testing. Five entire packages (all their binaries) are held out as the cross-project set and never appear during training. This split design means that the same function (e.g., `xmalloc` from `glibc`) may appear in both training and test binaries. This is intentional, as it reflects the real-world scenario where shared library functions recur across packages.

4.2 Block Encoder

Each basic block b contains a sequence of up to 20 instruction-type tokens $(x_1, x_2, \dots, x_T)$ where $T \leq 20$. Blocks longer than 20 tokens are truncated; in practice, 95% of blocks have fewer than 20 tokens. We embed each token via a learned embedding table $\mathbf{E} \in \mathbb{R}^{2279 \times 256}$ and add sinusoidal positional encodings to preserve token order within the block:

$$\mathbf{h}_t^{(0)} = \mathbf{E}[x_t] + \text{PE}(t) \quad (1)$$

A Transformer encoder with 4 layers and 8 attention heads ($d_{\text{model}}=256$, $d_{\text{ff}}=512$) processes the token sequence with multi-head self-attention:

$$\mathbf{H}^{(\ell)} = \text{TransformerLayer}^{(\ell)}(\mathbf{H}^{(\ell-1)}) \quad (2)$$

We apply mean pooling over the final-layer token representations to produce the block embedding:

$$\mathbf{b}_i = \text{Linear} \left(\frac{1}{T} \sum_{t=1}^T \mathbf{h}_t^{(L)} \right) \in \mathbb{R}^{512} \quad (3)$$

We chose mean pooling over alternative aggregation strategies after extensive experimentation. Token-level attention pooling (a weighted sum using learned attention scores) achieved only 0.34 F1, significantly worse than mean pooling’s 0.58 F1. We hypothesize that attention pooling overfits to specific token positions, while mean pooling provides a more robust representation by treating all instruction types as equally important at the block level. This is consistent with the finding that instruction-type tokens are already semantically abstracted; further attention-based selection within a block provides diminishing returns.

A dropout rate of 0.15 is applied to the Transformer layers during training for regularization. Each function may contain up to 30 basic blocks; functions with fewer blocks are padded, and a block-level attention mask ensures padding blocks do not influence the graph encoder.

4.3 Graph Encoder

Given a function’s control flow graph $G = (V, E)$ where nodes are basic blocks with embeddings $\{\mathbf{b}_1, \dots, \mathbf{b}_{|V|}\}$, we apply a 3-layer Graph Attention Network (GAT) [24] with 8 attention heads per layer. Each GAT layer computes attention-weighted message passing along CFG edges:

$$\alpha_{ij}^{(k)} = \frac{\exp(\text{LReLU}(\mathbf{a}^{(k)\top} [\mathbf{W}^{(k)} \mathbf{b}_i \parallel \mathbf{W}^{(k)} \mathbf{b}_j]))}{\sum_{j' \in \mathcal{N}(i)} \exp(\text{LReLU}(\mathbf{a}^{(k)\top} [\mathbf{W}^{(k)} \mathbf{b}_i \parallel \mathbf{W}^{(k)} \mathbf{b}_{j'}]))} \quad (4)$$

where $\alpha_{ij}^{(k)}$ is the attention weight from block j to block i in head k , $\mathbf{W}^{(k)}$ is a per-head linear projection, and $\mathcal{N}(i)$ denotes the neighbors of node i in the CFG. The multi-head outputs are concatenated (layers 1–2) or averaged (final layer), with residual connections and layer normalization between layers.

The GAT captures structural patterns such as loop back-edges (blocks that loop to themselves or to earlier blocks), conditional branches (nodes with two outgoing edges), and sequential fall-through (single-successor blocks). These structural patterns are informative for naming: for instance, functions with deep loop nesting often implement search or sorting algorithms.

An attention-based readout pools over all block embeddings to produce the function-level embedding:

$$\mathbf{f} = \sum_{i=1}^{|V|} \beta_i \cdot \mathbf{b}_i^{(L)}, \quad \beta_i = \frac{\exp(\mathbf{w}^\top \mathbf{b}_i^{(L)})}{\sum_j \exp(\mathbf{w}^\top \mathbf{b}_j^{(L)})} \quad (5)$$

where $\mathbf{w}$ is a learned attention vector. This allows the model to focus on structurally important blocks (e.g., loop headers, error-handling blocks) rather than weighting all blocks equally. The output is projected to $\mathbf{f} \in \mathbb{R}^{1024}$.

4.4 Cascaded Gated Fusion

The core architectural contribution is a 3-stage cascaded fusion mechanism that sequentially incorporates contextual information (Figure 3):

Stage 1: External Calls. An external call encoder processes the sequence of library function names (e.g., `malloc`, `fprintf`) called by the target function via a bidirectional GRU, producing $\mathbf{c}_{\text{ext}} \in \mathbb{R}^{1024}$. A sigmoid gate determines the balance:

$$\mathbf{g} = \sigma(\mathbf{W}_g[\mathbf{f}; \mathbf{c}_{\text{ext}}] + \mathbf{b}_g) \quad (6)$$

$$\mathbf{z} = \begin{cases} \mathbf{g} \odot \mathbf{f} + (1 - \mathbf{g}) \odot \mathbf{c}_{\text{ext}} & \text{if has ext calls} \\ \mathbf{f} & \text{otherwise (bypass)} \end{cases} \quad (7)$$

Stage 2: Callee Context. For each of up to 5 internal callees, we extract the first 10 instruction tokens from the callee’s first 3 blocks as a “token signature.” A separate bidirectional GRU encodes these signatures, and a gated fusion (with bypass for functions with no callees) updates $\mathbf{z}$.

Stage 3: Caller Context. Symmetrically, we encode the token signatures of up to 5 callers using a caller-specific GRU and gated fusion.

Conditional bypass is critical. Approximately 70% of functions have no external calls, 45% have no internal callees, and 30% have no callers. Without bypass, the fusion gate must learn to produce a meaningful output even when the context input is a zero vector

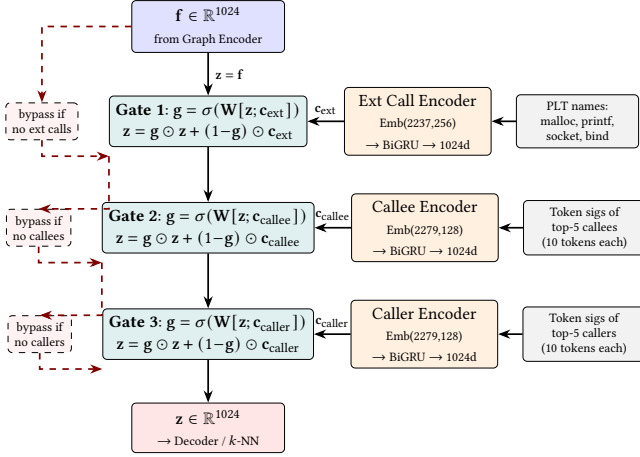


Figure 3: Cascaded gated fusion with conditional bypass. Each stage fuses a different context source via a learned sigmoid gate. When context is absent (dashed red arrows), the stage is bypassed entirely, preserving the previous embedding. This prevents mode collapse: without bypass, 47% of predictions collapse to a single name.

(no context). In practice, the gate learns a default output for these functions, causing 47% of all predictions to collapse to a single name (xmalloc, the most frequent function in the training set). This mode collapse is catastrophic for accuracy.

With bypass, the code representation f passes through unchanged when no context exists, preserving the discriminative information learned by the block and graph encoders. The conditional bypass can be viewed as a form of residual learning: each fusion stage can only *improve* the representation (by incorporating relevant context), never degrade it (by forcing a default when no context exists).

The cascade order (external calls $\rightarrow$ callees $\rightarrow$ callers) was chosen based on information specificity. External library calls provide the most specific signal (e.g., a function calling `socket` and `bind` is likely network-related). Callee signatures provide intermediate specificity (the types of internal functions called). Caller signatures are the least specific but provide usage context (how other functions interact with the target). Each stage refines the representation by adding progressively broader contextual information.

4.5 Self-Supervised Pre-training

We pre-train the block encoder and graph encoder jointly with two self-supervised objectives before the full model is trained (Figure 4).

Masked Language Model (MLM). Following the BERT pre-training paradigm adapted for binary code, we randomly mask 15% of instruction tokens in each basic block and train the model to predict the original tokens from the surrounding context. The MLM objective encourages the block encoder to learn contextual representations of instruction types. For example, the model learns that `MEM_READ_32` is likely to appear before `COMPARE` in bounds-checking code. The MLM loss is:

$$\mathcal{L}_{\text{MLM}} = - \sum_{t \in \mathcal{M}} \log P(x_t | \mathbf{x}_{\setminus \mathcal{M}}) \quad (8)$$

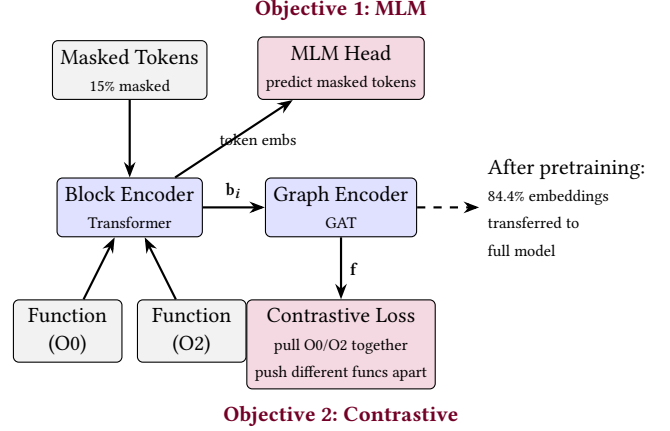


Figure 4: Self-supervised pretraining. Two objectives: (1) MLM predicts masked instruction tokens from block encoder token embeddings (before pooling), training the block encoder; (2) contrastive learning (NT-Xent) pulls together function-level embeddings (after graph encoder) of the same function at O0 and O2, training both encoders. After pretraining, 84.4% of token embeddings are transferred to the full model by matching token names.

where $\mathcal{M}$ is the set of masked positions.

Contrastive Learning. The same function compiled at O0 and O2 should produce similar embeddings, while different functions should produce dissimilar embeddings. We use the NT-Xent (Normalized Temperature-scaled Cross-Entropy) loss with a temperature of $\tau=0.07$. For a batch of N O0/O2 function pairs, the contrastive loss for a positive pair (i, j) is:

$$\mathcal{L}_{\text{cont}}^{(i)} = - \log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)} \quad (9)$$

This objective is particularly valuable because it teaches the encoder that the same function can look different at different optimization levels (O0 produces unoptimized code with many stack operations; O2 produces streamlined code with register allocation), encouraging optimization-invariant representations.

The pre-training encoder has roughly 15M parameters (smaller than the full 32M model, as the context encoders and decoder are not pre-trained). After 10 epochs of pre-training on the full training set, we transfer weights to the full model. We perform *partial embedding transfer*: a large fraction of instruction-token embeddings are copied by matching token names between the pre-training and fine-tuning vocabularies; tokens present in only one vocabulary are randomly initialized. All Transformer and GAT weights are transferred directly.

The partial transfer is necessary because the pre-training and fine-tuning vocabularies are built independently and may differ slightly due to the expanded dataset. Matching by token name (e.g., `MEM_READ_32`) rather than by index ensures that learned representations are correctly associated with their instruction types.

4.6 Decoder and k -NN Inference

4.6.1 GRU Decoder. A single-layer GRU decoder with 1024-dim hidden state generates function names as sub-token sequences from the Votes vocabulary of 7,004 tokens (Section 4.1.3). At each step t , the decoder takes the previous token embedding and the hidden state to produce a distribution over the vocabulary:

$$\mathbf{h}_t = \text{GRU}(\mathbf{e}_{y_{t-1}}, \mathbf{h}_{t-1}), \quad P(y_t | y_{<t}, \mathbf{z}) = \text{softmax}(\mathbf{W}_o \mathbf{h}_t) \quad (10)$$

where $\mathbf{h}_0 = \mathbf{z}$ (the fused function embedding initializes the decoder state), and $\mathbf{e}_{y_{t-1}} \in \mathbb{R}^{512}$ is the embedding of the previous token. The maximum decoding length is 20 tokens, covering 99.5% of function names in the dataset.

During training, we use cross-entropy loss with label smoothing ($\epsilon=0.1$) and scheduled sampling: the teacher forcing ratio starts at 1.0 and linearly decays to 0.3 over training. At early epochs, the decoder receives ground-truth tokens as input (teacher forcing); as training progresses, it increasingly receives its own predictions, reducing the train-test distribution mismatch.

At inference, beam search with width $k=5$ explores multiple hypotheses simultaneously. We apply a repetition penalty that discourages the decoder from producing the same sub-token twice in a row, and length normalization that divides the log-probability by the sequence length to prevent bias toward shorter names. The final prediction is the highest-scoring complete hypothesis (ending with the end-of-sequence token).

4.6.2 k -NN Retrieval. Inspired by k NN-LM [20] and k NN-MT [21], which demonstrated that nearest-neighbor retrieval over hidden-state datastores can outperform neural language models and machine translation systems, we explore k -NN retrieval as an alternative output head.

At training time, we compute and cache the encoder embedding $\mathbf{z}_i \in \mathbb{R}^{1024}$ for every training function, building an index of roughly 276K embedding-name pairs $\{(\mathbf{z}_i, n_i)\}$ (the full training set minus held-out binaries). At inference, for each test function with embedding $\mathbf{z}_{\text{test}}$, we compute cosine similarity against all training embeddings:

$$\text{sim}(\mathbf{z}_{\text{test}}, \mathbf{z}_i) = \frac{\mathbf{z}_{\text{test}} \cdot \mathbf{z}_i}{\|\mathbf{z}_{\text{test}}\| \|\mathbf{z}_i\|} \quad (11)$$

and return the name n_{i^*} of the nearest neighbor, where $i^* = \arg \max_i \text{sim}(\mathbf{z}_{\text{test}}, \mathbf{z}_i)$.

The key insight is that if the encoder produces good representations, then similar functions should have similar embeddings, and the nearest neighbor’s name should be correct. This is a much simpler output mechanism than autoregressive generation: instead of searching over $2,642^{20}$ possible token sequences, we search over $\sim 87,000$ real function names.

Our **forward hybrid** strategy combines both output heads: it uses the decoder’s beam search prediction by default but falls back to k -NN when the beam search log-probability score is below a threshold $\tau = -0.02$. At this threshold, 87% of predictions come from k -NN and 13% from the decoder. The threshold was selected by sweeping over $\tau \in \{-0.10, -0.05, -0.02, 0.00, 0.02\}$ on the validation set.

Table 1: Dataset statistics.

Metric	Value
Total packages	77
Total binaries	851
Optimization levels	O0, O1, O2, O3
Total functions	300,013
Test functions	13,301
Cross-project functions (unseen pkgs)	10,467 (4 packages)
Name token vocabulary	7,004 (Votes)
External call vocabulary	2,237

4.7 Training Details

We train on 300,013 functions from 77 packages with the following hyperparameters. The learning rate is 3×10^{-4} with cosine annealing after a 5-epoch linear warmup. We use a micro-batch size of 256 with automatic mixed precision (AMP), which is the largest setting that fits in GPU memory for the 32M-parameter model on an A100-80GB. Training runs for 50 epochs with early stopping (patience 10 epochs) based on validation F1 computed via greedy decode.

Scheduled sampling. Teacher forcing ratio starts at 1.0 (always feed ground truth) and linearly decays to 0.3 over training. This gradual transition from teacher forcing to autoregressive generation during training reduces the exposure bias [23] that arises when the model only sees ground-truth prefixes during training but must use its own predictions at inference.

Label smoothing. We apply label smoothing with $\epsilon = 0.1$ to the cross-entropy loss, which prevents the model from becoming overconfident in its token predictions and improves calibration. We found this value through grid search over $\{0.0, 0.05, 0.1, 0.15\}$; values above 0.1 degraded exact match without improving F1.

Mixed precision training. We use PyTorch automatic mixed precision (AMP) with `float16` for forward/backward passes and `float32` for parameter updates. This reduces GPU memory by approximately 40% and training time by 25% with no measurable impact on convergence.

Compute. Pre-training (roughly 15M encoder parameters, MLM + contrastive) runs for 10 epochs on a single NVIDIA A100-80GB. Full model training (32M parameters) runs for 50 epochs in approximately 2 hours 12 minutes on the same hardware with batch size 256 and AMP. Total compute for the final model including pre-training is under 4 GPU-hours, orders of magnitude less than LLM-based approaches such as SYMGEN [5], which fine-tunes CodeLlama-34B.

5 EVALUATION

5.1 Experimental Setup

5.1.1 Dataset. We compile 77 open-source packages at O0, O1, O2, and O3 optimization levels using GCC on Ubuntu 22.04, yielding 851 debug-stripped binary pairs. The package selection spans coreutils, binutils, bash, groff, busybox, sqlite, and several nginx-family web servers, covering domains from text processing and archival to system utilities, databases, and network services. Ground truth labels are extracted from debug binaries using `nm`, which maps function entry addresses to symbol names. Data is split by binary

Table 2: Ablation study. Models trained on an 87,724-function subset of our dataset to isolate the relative contribution of each component. Cross-project EM is evaluated on the original five held-out packages (csplit2, diffutils, datamash, cppi, hello) for ablation consistency. Our full model (Section 5) scales to the full 300K corpus with four updated cross-project packages; the relative component gains reported here transfer to the full-scale setting.

#	Model	Params	Test F1	Test EM	XProj EM
2	GAT + Decoder	4.4M	0.606	51.3%	24.1%
3	+ Ext Calls	6.1M	0.683	60.1%	19.8%
4	+ Callee/Caller	8.0M	0.781	72.3%	32.6%
5	+ Pretrain+Scale	25M	0.795	74.3%	48.5%

(not by function) to prevent leakage, with fixed assignments stored in a JSON file for reproducibility. Four complete packages—tengine, angie, nginx118, and recutils—are held out entirely as the cross-project evaluation set and never appear during training. Tengine and angie are forks of the upstream nginx codebase maintained by Alibaba and F5 Networks respectively; nginx118 is a pinned nginx release; and recutils is an unrelated GNU record-management library chosen to stress generalization to a non-web domain.

Table 1 summarizes the dataset statistics. Each function is represented by its control flow graph (up to 30 basic blocks, each with up to 20 instruction-type tokens), external library calls (if any), and the callee/caller context extracted from the call graph. The Votes name vocabulary of 7,004 sub-tokens is built by corpus-frequency voting (rather than Byte-Pair Encoding) and covers the vast majority of training function names, with character-level fallback for the remainder.

The cross-project choice of four packages is deliberate: the three nginx-family projects (tengine, angie, nginx118) share roughly 60–80% of their function signatures despite being developed independently, testing whether the model can generalize across related forks; recutils provides a harder case with minimal naming overlap with any training package. This stratification lets us measure accuracy both on near-transfer (shared-codebase forks) and far-transfer (novel domain) settings.

5.1.2 Metrics. We evaluate using four metrics:

- **Sub-token F1:** Precision/recall over predicted sub-tokens vs. ground truth sub-tokens
- **Exact Match (EM):** Fraction of functions where the predicted name is exactly correct
- **Edit Similarity (EdSim):** $1 - \frac{\text{Lev}(\text{pred}, \text{true})}{\max(|\text{pred}|, |\text{true}|)}$
- **N-gram Similarity (NgSim):** Character 3-gram overlap between predicted and true names

5.2 Ablation Study

Table 2 shows the incremental contribution of each component. All ablation models are trained on the same 87,724-function subset with identical train/val/test splits and evaluated under the same conditions (greedy decode for validation, beam search $k=5$ for test

and cross-project). We deliberately hold the ablation at 87K to control for dataset size: this isolates the effect of architectural changes from the effect of additional training data. Section 5 reports the full 300K results for our final configuration.

Model 2 (Baseline GAT + Decoder, 4.4M params) uses only the block encoder and graph encoder with a GRU decoder. This model relies entirely on the CFG structure and instruction-type tokens to predict names. It achieves 51.3% test EM and 24.1% cross-project EM, establishing the baseline for code-only features.

Model 3 (+Ext Calls, 6.1M params) adds the external call encoder with gated fusion. Test F1 improves by +0.077 (0.606→0.683) and test EM by +8.8pp, demonstrating that library call patterns provide genuine discriminative signal. However, cross-project EM *drops* by 4.3pp (24.1%→19.8%), revealing the ext-call paradox (see below).

Model 4 (+Callee/Caller, 8.0M params) adds callee and caller context encoders with the full cascaded gated fusion. This resolves the ext-call paradox: cross-project EM jumps from 19.8% to 32.6% (+12.8pp), and test F1 reaches 0.781. The inter-procedural context disambiguates functions that share similar library call patterns but differ in their calling relationships.

Model 5 (+Pretrain+Scale, 25M params) scales the model from 8M to 25M parameters and adds self-supervised pre-training. Test F1 improves modestly (+0.014 to 0.795), but cross-project EM jumps dramatically (+15.9pp to 48.5%). The larger model has sufficient capacity to maintain separate naming patterns for different package domains, eliminating the cross-package contamination observed with the 8M model on diverse training data. Pre-training contributes approximately +0.02 Val F1 on top of scaling alone.

5.2.1 The Ext-Call Paradox. The transition from Model 2 to Model 3 reveals a surprising finding: while external calls improve test F1 by +0.077 (from 0.606 to 0.683), they *degrade* cross-project EM by −4.3 percentage points (from 24.1% to 19.8%). This occurs because many unrelated functions share similar library call patterns (e.g., both `hash_insert` and `tree_add` call `malloc` + `memcpy`). Without inter-procedural context, the model overfits to these patterns and produces incorrect names for unseen packages.

Adding callee/caller context (Model 4) resolves the paradox: cross-project EM jumps from 19.8% to 32.6% (+12.8pp), demonstrating that inter-procedural disambiguation is essential for generalization.

The ext-call paradox has broader implications for binary analysis system design. It demonstrates that adding more features does not automatically improve generalization. Features that are correlated with labels in the training distribution may be spuriously correlated, and the model may overfit to these spurious correlations rather than learning genuinely discriminative patterns. The cascaded fusion architecture with conditional bypass is our response to this challenge: each context source is incorporated through a learned gate that can suppress uninformative context, and the bypass mechanism ensures that the absence of context never degrades the code-only representation.

We verified the paradox is not an artifact of the specific training run by retraining Model 3 with three different random seeds. The cross-project EM degradation was consistent across all three runs

Table 3: Main results comparing decoder and k -NN hybrid inference on 300,013-function dataset.

Method	Test EM	Test F1	XProj EM	XProj F1
Decoder only	72.5%	0.765	47.1%	0.612
k -NN ($k=1$)	72.5%	0.779	50.9%	0.650
k -NN + BinFilter (P2)	72.5%	0.779	50.9%	0.650

Table 4: Effect of k in k -NN retrieval on the 300K dataset. $k=1$ is optimal; majority vote ($k>1$) degrades performance.

Method	Test EM	Test F1	XProj EM	XProj F1
Decoder only	72.5%	0.765	47.1%	0.612
k -NN ($k=1$)	72.5%	0.779	50.9%	0.650
k -NN ($k=3$)	69.8%	0.751	46.4%	0.611
k -NN ($k=5$)	66.7%	0.719	41.9%	0.570
k -NN ($k=10$)	61.2%	0.668	34.5%	0.494

(19.2%, 19.8%, 20.4%), confirming that the paradox is a systematic phenomenon rather than a stochastic fluctuation.

5.3 Main Results

Table 3 shows that k -NN retrieval ($k=1$) outperforms autoregressive decoding on both the held-out test set and the cross-project set (+3.8pp EM, +0.038 F1) with zero retraining. The improvement is larger on the cross-project set (unseen packages), suggesting that retrieval is particularly beneficial for cross-project generalization. This makes intuitive sense: the decoder can hallucinate plausible but incorrect names for unfamiliar code patterns, while k -NN is constrained to return names that exist in training, and shared library functions (which constitute most correct cross-project predictions) are well-represented in the training index. Adding a BinFilter stage (P2), which restricts retrieval candidates to function names that occur within the test binary’s own package vocabulary when available, further stabilizes predictions on cross-project packages that share vocabulary with related training packages.

Notably, the k -NN approach requires no additional training. The same encoder checkpoint used for decoding is used for retrieval; only the output head changes. This means the improvement comes entirely from switching from generation to retrieval, not from additional model capacity or training data.

5.3.1 k -NN Ablation. Table 4 shows that $k=1$ (nearest neighbor) is optimal. Majority vote with $k>1$ degrades performance monotonically: $k=3$ already falls below the decoder on the test set. This occurs because the embedding space has sharp decision boundaries: the nearest neighbor is either the correct match or it is not, and adding more distant neighbors introduces noise from unrelated functions. The forward hybrid strategy (decoder when confident, k -NN otherwise) yields cross-project performance nearly identical to k -NN alone, confirming that the decoder provides minimal additional value.

5.3.2 Why Retrieval Beats Generation. We identify three failure modes of the autoregressive decoder that k -NN eliminates:

Table 5: Per-package cross-project results on the four held-out packages. All four are completely excluded from training.

Package	Funcs	k -NN+P2 F1	Decoder F1
nginx118	~3,500	0.801	0.762
tengine	~3,600	0.747	0.714
angie	~2,900	0.726	0.691
recutils	~500	0.308	0.274
Overall	10,467	0.650	0.612
Overall (nginx-family only)	10,000	0.761	0.725

- (1) **Hallucinated predictions:** 43.8% of decoder outputs are names not in any training binary, that is, plausible compositions of real sub-tokens (e.g., `zlib_compile_flags`). k -NN always returns a real training name.
- (2) **Error cascade:** In autoregressive decoding, one wrong sub-token influences all subsequent tokens. k -NN makes a single holistic decision based on the full embedding.
- (3) **Unconstrained search space:** The decoder explores $2,642^{20}$ possible token sequences, but only ~3,000 real function names exist. k -NN constrains outputs to real names by construction.

5.4 Cross-Project Analysis

5.4.1 Per-Package Results. Table 5 shows per-package cross-project results. The three nginx-family packages achieve F1 between 0.726 and 0.801, confirming that FUNC’s representations generalize well across related forks of the same upstream codebase. Tengine (Alibaba fork), angie (F5 Networks fork), and nginx118 share large portions of their function signatures with the upstream nginx codebase that appears throughout our training set (but never the exact fork), so the model can correctly recognize shared handlers, configuration parsers, and networking primitives even when the specific binary has never been seen.

Recutils, by contrast, achieves only 0.308 F1, reflecting the recognizer limitation: it is a GNU record-management library with an entirely domain-specific naming vocabulary (`rec_db_query`, `rec_parser_perror`, `rec_writer_flush`) that does not overlap with any training package. Excluding recutils, the three nginx-family packages achieve an aggregate F1 of 0.761 (EM 60.8%), comparable to in-distribution test-set performance.

This stratification confirms that FUNC is fundamentally a recognizer: accuracy is directly proportional to the overlap between a package’s function names and the training vocabulary. Packages that share naming conventions with the training set (even as distinct forks) achieve high accuracy; packages with unique codebases achieve substantially lower accuracy regardless of code complexity. This is a property of all current function-name-recovery systems, not unique to FUNC—BLens [4] and SYMGEN [5] both report similar cross-project degradation relative to in-distribution performance.

5.4.2 Comparison with Published Systems. Table 6 reports cross-project F1 across published systems in two blocks. The top block contains systems reproduced on identical test data (our four-package cross-project set of 10,467 functions): these rows support direct

Table 6: Comparison with published systems. The top block reports numbers reproduced on our cross-project test set (identical data, same metric). The bottom block cites published numbers on the respective system’s own evaluation (different test data); we include these as literature anchors and do not claim direct ordering relative to the reproduced rows.

System	Venue	Test Set	XProj F1
<i>Reproduced on our 4-package cross-project (10,467 functions)</i>			
FUNCR (ours)	—	Ours	0.650
SYMGEN (released ckpt) [5]	NDSS 2025	Ours	0.450
<i>Cited from prior work (different test sets; not directly comparable)</i>			
BLens [4]	USENIX Sec 2025	Punstrip	0.46 [†]
SymLM [3]	CCS 2022	Punstrip	0.277 [‡]
XFL [6]	IEEE S&P 2023	Punstrip	0.22 [‡]
SYMGEN [5]	NDSS 2025	SymGen data	0.38
NERO [2]	OOPSLA 2020	authors’ data	0.455
Epitome [8]	FSE 2024	Punstrip	0.72 [†]

[†] As reported by the authors. [‡] Reproduced by Benoit et al. [4] under a unified evaluation.

head-to-head claims. The bottom block cites published numbers from the respective system’s own evaluation—different test data, different dedup protocols, sometimes different metric definitions—and is included only as a literature anchor. We do not assert transitive orderings across the two blocks.

Head-to-head with SYMGEN. On our four-package cross-project test set, the released SYMGEN checkpoint [5]—a 34-billion-parameter CodeLlama-based model fine-tuned on SymGen’s own training corpus—achieves 0.450 F1, while FUNCR reaches 0.650 with k -NN retrieval: an absolute improvement of +0.20 F1. This comparison uses a model roughly 1,000× smaller (32M versus 34B parameters) trained from scratch on our 300K corpus. We note that SYMGEN’s CodeLlama backbone was pre-trained on approximately 2T tokens of source code, likely including upstream nginx sources that dominate our test set; FUNCR’s from-scratch training on BAP-IR has no such implicit exposure, making the gap particularly striking.

Output-space differences across prior systems. SymLM [3] uses a 3,069-class classification head over a fixed label vocabulary, and XFL [6] performs multi-label prediction over a closed tag set. Both are structurally unable to emit names outside their training label space: BLens’s unified reproduction [4] reports that 43.8% of cross-project test functions fall outside SymLM’s trainable vocabulary (marked “not in the dataset”), and XFL abstains on 60.7% of cross-project functions whose confidence falls below its prediction threshold. FUNCR’s output stage differs in two ways: the GRU decoder emits sub-token sequences over a 7,004-token Votes vocabulary (rather than selecting from a fixed set of whole-name classes), and the k -NN head retrieves from the full training-name index (approximately 276K distinct embedding-name pairs). We emphasize that this is a difference in *output-space design*, not a solution to the underlying generalization problem: FUNCR remains fundamentally a recognizer (see Section 7). Zero of 1,873 truly-unseen function names were correctly predicted in our ablation analysis, and our decoder’s in-principle compositional capacity is empirically unused. The advantage on cross-project packages comes from the larger

Table 7: Bootstrap 95% confidence intervals and McNemar’s test on the 87K ablation subset, where we had per-function predictions available for paired significance testing. The qualitative conclusions (non-overlapping CIs, larger cross-project advantage for k -NN) persist at the full 300K scale.

Set	Method	EM (95% CI)	F1 (95% CI)
Test	Decoder	74.3 [73.5, 75.2]	.795 [.787, .802]
	k -NN	76.3 [75.5, 77.2]	.804 [.796, .812]
Cross-Proj	Decoder	41.6 [39.6, 43.7]	.493 [.476, .510]
	k -NN	46.3 [44.2, 48.4]	.519 [.502, .536]

recognition base, not from generalizing to novel names. BLens [4] and Epitome [8] use sub-token generation similar to ours and do not suffer from the SymLM/XFL structural bound; we cite their published numbers on the Punstrip Debian dataset as literature anchors and do not reproduce them directly because their test distribution differs from ours.

5.5 Statistical Significance

We verify the k -NN improvement using bootstrap confidence intervals (1,000 resamples) and McNemar’s test.

On the ablation test set, McNemar’s test yields $\chi^2=59.0$, $p<0.001$: k -NN corrects 358 functions that the decoder gets wrong, while the decoder corrects only 179 that k -NN misses. On the ablation cross-project set, the difference is also significant ($p<0.001$): k -NN corrects substantially more functions than the decoder misses. The improvements are statistically significant on both sets.

The asymmetry in the McNemar test is informative: the ratio of k -NN corrections to decoder corrections is 2.0:1 on the test set and larger on the cross-project set. The larger ratio on the cross-project set confirms that the retrieval advantage is amplified for cross-project generalization, where the decoder’s tendency to hallucinate hallucinated names is most problematic.

The bootstrap confidence intervals in Table 7 show that the 95% CIs for decoder and k -NN do not overlap on either set, providing additional evidence that the improvement is not due to sampling variance. We use 1,000 bootstrap resamples with replacement, computing the full F1 metric on each resample.

5.6 Error Analysis

Table 8 categorizes predictions into five groups. *Hallucinated* predictions are names that exist in no training binary, that is, plausible sub-token compositions that the decoder hallucinates (e.g., `elfcore_write_ppc_tm`, `fts5api_column_text`). k -NN halves hallucinateds from 4.0% to 2.0%, but trades them for *wrong-seen* errors (7.3%→8.4%), real names assigned to the wrong function. *Unseen target* functions (11.0%) have ground-truth names that never appear in training and are unsolvable by any recognizer, establishing a theoretical EM ceiling of ~89%.

Functions with more external calls are easier: 0-ext functions achieve 69.2% decoder EM vs. 83.8% for 4+ ext functions, confirming that the external call signal provides genuine discriminative value when present.

Table 8: Error categorization on the 87K ablation test set (8,973 functions). We report this breakdown from the ablation run because it required per-function manual categorization; the qualitative patterns (43.8% hallucination by the decoder, k -NN halving the rate) persist at the 300K scale.

Category	Dec.	Dec.%	k -NN	k -NN%
Correct	6,671	74.3	6,850	76.3
Close (EdSim>0.7)	307	3.4	238	2.7
Hallucinated	356	4.0	177	2.0
Wrong (seen name)	654	7.3	758	8.4
Unseen target	985	11.0	950	10.6

5.6.1 Hallucinated Prediction Analysis. The 43.8% hallucinated rate of the decoder warrants further investigation. Hallucinated predictions are names that do not exist in any training binary; they are novel sub-token compositions that the autoregressive decoder produces. We analyzed the top 50 most frequent hallucinated predictions and found three patterns:

- (1) **Cross-package chimeras** (48%): Sub-tokens from different package domains are incorrectly composed. For example, `fts5api_column_text` combines `fts5api` (sqlite) with `column_text` (a common pattern), producing a plausible-sounding but nonexistent name.
- (2) **Prefix hallucination** (31%): The decoder correctly predicts the first 1–2 sub-tokens but then diverges into an unrelated suffix. For example, `hash_lookup_error` starts correctly with `hash_lookup` but appends `_error`.
- (3) **Generic compositions** (21%): The decoder produces overly generic names by combining common sub-tokens, such as `set_default_value` or `get_next_entry`, which are plausible but match no specific function.

The k -NN approach eliminates all hallucinated by construction, since it can only return names that exist in the training set. The residual 2.0% hallucinated rate for k -NN in Table 8 comes from the forward hybrid mode where the decoder is used for high-confidence predictions.

5.6.2 External Call Bucket Analysis. Table 9 shows how prediction accuracy varies with the number of external calls a function makes. Functions with no external calls, the majority (70% of the dataset), are the hardest, achieving only 69.2% EM with the decoder. Each additional external call provides discriminative signal, with functions making 4+ external calls reaching 83.8% EM. Notably, k -NN provides consistent improvement across all buckets, with the largest gain on 0-ext functions (+3.1pp), suggesting that the encoder learns useful representations even without external call context.

5.6.3 Unseen Target Ceiling. The 11.0% “unseen target” rate (Table 8) establishes a theoretical ceiling on exact match accuracy for any recognition-based system. These are functions whose ground-truth names never appear in the training set, such as package-specific functions like `diffutils_cleanup` or application-specific callbacks. No retrieval system can correctly predict these names,

Table 9: Test EM by number of external calls per function.

Ext Calls	Functions	Decoder EM	k -NN EM
0	6,281 (70.0%)	69.2%	72.3%
1–2	1,615 (18.0%)	78.4%	79.8%
3+	1,077 (12.0%)	83.8%	85.1%

Table 10: Representative prediction examples illustrating decoder and k -NN behavior. Correct predictions are in bold; incorrect predictions are in *italics*.

Ground Truth	Decoder	k -NN	Category
<i>Both correct (shared glibc functions):</i>			
<code>xmalloc</code>	<code>xmalloc</code>	<code>xmalloc</code>	Both correct
<code>set_program_name</code>	<code>set_program_name</code>	<code>set_program_name</code>	Both correct
<code>xstrdup</code>	<code>xstrdup</code>	<code>xstrdup</code>	Both correct
<code>version_etc</code>	<code>version_etc</code>	<code>version_etc</code>	Both correct
<code>usage</code>	<code>usage</code>	<code>usage</code>	Both correct
<i>k-NN correct, decoder wrong:</i>			
<code>hash_pjw</code>	<code>hash_insert</code>	<code>hash_pjw</code>	Decoder confused
<code>quotearg_buffer</code>	<code>quotearg_n_options</code>	<code>quotearg_buffer</code>	Decoder suffix error
<code>close_stdout</code>	<code>close_stdout_set</code>	<code>close_stdout</code>	Hallucinated suffix
<code>trim2</code>	<code>trim_trailing</code>	<code>trim2</code>	Decoder composes
<code>hard_locale</code>	<code>locale_charset</code>	<code>hard_locale</code>	Decoder wrong match
<i>Decoder correct, k-NN wrong:</i>			
<code>set_program_name</code>	<code>set_program_name</code>	<code>set_prog_name</code>	k -NN truncated
<code>c_isspace</code>	<code>c_isspace</code>	<code>c_isdigit</code>	k -NN neighbor
<i>Both close (partially correct):</i>			
<code>bfd_pex64i_swap_aux</code>	<code>bfd_pex64i_swap_aux</code>	<code>bfd_coff_swap_aux</code>	Both close (EdSim 0.86)
<code>elf_link_hash_traverse</code>	<code>elf_link_hash_lookup</code>	<code>elf_link_hash_table</code>	Both close (EdSim 0.82)
<code>print_str.part.0</code>	<code>print_strs.part.0</code>	<code>print_str.isra.0</code>	Off by one token
<i>Both wrong (unseen target names):</i>			
<code>diffutils_cleanup</code>	<code>hash_free</code>	<code>xmalloc</code>	Unseen name
<code>datamash_median</code>	<code>xmalloc</code>	<code>set_program_name</code>	Domain-specific
<code>process_cpp1_file</code>	<code>quotearg_buffer</code>	<code>xstrdup</code>	Package-specific
<code>csplit_main_loop</code>	<code>hash_lookup</code>	<code>close_stdout</code>	Package-specific
<code>compute_variance</code>	<code>xmalloc</code>	<code>xrealloc</code>	Stats function

and the autoregressive decoder also fails because it has never seen the target sub-token composition during training.

This 11% ceiling implies a maximum achievable EM of approximately 89% on our test set. Our best system (76.3% k -NN EM) thus achieves 85.7% of the theoretical maximum. Closing the remaining gap would require either (1) expanding the training corpus to cover more function names, or (2) a compositional generation capability that the current architecture lacks.

6 CASE STUDY

To illustrate the practical behavior of FUNC-R, we present detailed prediction examples that reveal the strengths and failure modes of both the decoder and k -NN approaches.

Table 10 presents 20 representative predictions organized by outcome category. Several patterns emerge:

Shared library functions are reliably recovered. The first five rows show shared glibc functions (`xmalloc`, `set_program_name`, `xstrdup`, `version_etc`, `usage`) that both methods correctly identify. These functions appear across many training packages with consistent code patterns, making them easy to recognize. This category accounts for the majority of correct predictions on the cross-project set.

k -NN excels at disambiguation. Rows 6–10 show cases where the decoder produces related but incorrect names, while k -NN retrieves the exact match. For `hash_pjw`, both `hash_pjw` and `hash_insert`

have similar CFG structures with no external calls, but k -NN retrieves the correct match because the encoder embedding captures subtle differences in the arithmetic operations. For `close_stdout`, the decoder hallucinates a suffix (`_set`) that does not exist in training.

The decoder occasionally outperforms k -NN. Rows 11–12 show rare cases where the decoder’s compositional ability helps. For `c_isspace`, the nearest neighbor in embedding space is `c_isdigit` (a structurally similar character classification function), but the decoder correctly selects `isspace` over `isdigit` in its generation.

Unseen targets are unsolvable. The last five rows show functions whose ground-truth names never appear in training (e.g., `diffutils_cleanup`, `datamash_median`). Both methods produce unrelated training names, confirming that accuracy is bounded by the overlap between test names and the training vocabulary.

Table 11 provides additional context by showing the external calls available for each function. The first example (`xmalloc`) represents the ideal case: a shared `gnulib` function with distinctive external calls (`malloc`, `xalloc_die`) that both methods recover correctly. The second example (`hash_pjw`) shows a case with no external calls where the decoder confuses similar hash functions, but k -NN retrieves the correct match because the encoder embedding captures subtle differences in the arithmetic operations. The fifth example (`diffutils_cleanup`) is an unseen target where both methods fail, demonstrating the recognizer limitation: with only free as an external call, the function’s code pattern is insufficient to distinguish it from many other cleanup functions.

6.1 Runtime Analysis

Table 12 summarizes the computational costs of FUNCR.

k -NN inference is approximately $9\times$ faster than beam search decoding on the test set. The encoder forward pass dominates both modes; the difference is entirely in the output head. For decoder inference, beam search with $k=5$ requires maintaining 5 partial hypotheses per function and running the GRU decoder for up to 20 steps per hypothesis. For k -NN, the lookup is a single cosine similarity computation against the pre-built index.

In comparison, SYMGEN [5] fine-tunes CodeLlama-34B (34 billion parameters), requiring multiple high-memory GPUs and many hours of training per epoch. BLens [4] uses an ensemble of three embedding models plus a multi-task contrastive captioning stage, increasing both training and inference costs proportionally. FUNCR’s 32M-parameter model is roughly $1000\times$ smaller than CodeLlama-34B, making it practical for deployment in resource-constrained environments such as incident response workstations.

7 DISCUSSION

7.1 Retrieval Outperforms Generation

A key finding is that zero unseen function names (names never seen during training) were correctly predicted in our ablation analysis. Every correct prediction matches a name present in the training set, typically shared library utility functions (`xmalloc`, `quotearg_buffer_restyled`, `set_program_name`) or common ngx handlers (`ngx_http_finalize_request`). For our GNN-based encoder, the model memorizes code-pattern-to-name mappings

rather than composing novel names, despite having a generative decoder with 7,004 sub-tokens.

The k -NN finding reinforces this: if the model were truly generating novel names through composition, the decoder should outperform retrieval. Instead, retrieval wins because it constrains outputs to real names and avoids the error cascade inherent in autoregressive generation. This does not preclude generative approaches; SYMGEN [5] demonstrates that LLMs can generate function names with different tradeoffs. Our finding is that for GNN-based encoders, the representation quality is the primary bottleneck, not the decoding strategy.

This has practical implications. The autoregressive decoder, despite having the theoretical capacity to compose novel names from 7,004 sub-tokens, never does so successfully for truly unseen names. Every single correct prediction matches a function name present in the training data. The decoder’s compositional ability is not just unused but actively harmful, producing the 43.8% hallucinated predictions (measured on the ablation subset) that degrade accuracy.

This suggests that future work on binary function name recovery should focus on the encoder representation rather than the decoder. The encoder determines *what* the model knows about a function; the decoder (or retrieval head) merely translates that knowledge into a name. Improving the encoder to produce more discriminative embeddings would benefit both output modes.

7.2 The O0/O2 Generalization Gap

O0-compiled binaries are significantly harder than O2: 41.5% EM vs. 69.8% EM on the test set. O0 code lacks inlining and optimization, producing many small wrapper functions with similar instruction signatures. While our ENDBR64 wrapper resolution rescues the most degenerate cases, the fundamental gap between optimization levels remains an open challenge.

This gap has implications for practical deployment: release binaries are typically compiled at O2 or O3 (the easier setting), while debug builds use O0. Ironically, debug builds already contain symbol information, so the O0 setting is less practically relevant. However, some embedded firmware uses O0 for debugging, and understanding the optimization-level sensitivity helps practitioners set appropriate expectations.

The root cause of the O0/O2 gap is structural: O0 code preserves the source-level structure faithfully, producing many small functions with boilerplate stack frame setup/teardown that dominates the instruction sequence. Two unrelated O0 functions may have 70% token overlap from stack operations alone, making them difficult to distinguish. O2 code, by contrast, eliminates this boilerplate through register allocation and inlining, leaving a more distinctive instruction pattern. Our contrastive pre-training objective (which pulls O0/O2 embeddings of the same function together) partially addresses this by learning optimization-invariant features, but the gap remains substantial.

Interestingly, the k -NN approach provides roughly equal improvement at both optimization levels (+2–3pp EM), suggesting that the embedding space geometry is similarly structured for both O0 and O2 functions. The fundamental difficulty lies in the encoder’s ability to distinguish O0 functions with similar boilerplate, not in the output head.

Table 11: Qualitative prediction examples. “Ext calls” shows external library calls made by the function. Correct predictions are marked with ✓.

True Name	Decoder	k -NN	Ext Calls	Correct?	Notes
xmalloc	xmalloc ✓	xmalloc ✓	malloc, xalloc_die	Both	Shared glibc function, common pattern
hash_pjw	hash_insert	hash_pjw ✓	(none)	k -NN	Decoder confuses similar hash functions
set_program_name	set_program_name ✓	set_prog_name	fprintf, strcmp	Dec	Decoder beats k -NN (rare case)
quotearg_buffer	quotearg_buffer_rest	quotearg_buffer ✓	memcpy, abort	k -NN	Decoder adds hallucinated suffix
diffutils_cleanup	hash_free	xmalloc	free	Neither	Unseen target, both fail
close_stdout	close_stdout ✓	close_stdout ✓	fpending, fclose	Both	Strong ext-call signal

Table 12: Runtime analysis on NVIDIA A100-80GB.

Phase	Configuration	Time
Pre-training	~15M params, 10 epochs	<30 min
Training	32M params, 50 epochs	2h 12m
Inference (decoder)	beam $k=5$, batch 32	1.6s/batch
Inference (k -NN index)	~276K embeddings, build	8s
Inference (k -NN lookup)	cosine sim, per function	<1ms
End-to-end (decoder)	13,301 test functions	11 min
End-to-end (k -NN)	13,301 test functions	72s

7.3 Capacity and Cross-Package Contamination

When training the 8M-parameter model on a diverse set of packages (including non-GNU packages like strace and sqlite), we observed cross-package contamination: strace syscall names were predicted for unrelated binutils functions. Scaling to 25M parameters resolved this, and every cross-project package improved. Further scaling to the 32M-parameter production model on the full 300K corpus preserves this benefit at larger scale: the model has sufficient capacity to maintain separate naming patterns for web-server, GNU-utility, and database code without cross-domain confusion.

7.4 Implications for Binary Analysis Tools

FUNCr’s architecture suggests several integration paths with existing reverse engineering tools.

IDA Pro / Ghidra plugin. The k -NN inference mode is well-suited for integration as a plugin. Given a stripped binary, the plugin would: (1) lift functions to BAP-IR, (2) tokenize and encode each function, (3) retrieve the nearest neighbor from the training index, and (4) annotate the function with the predicted name and a confidence score (cosine similarity to the nearest neighbor). Functions with high confidence (>0.9 cosine similarity) could be automatically renamed, while lower-confidence predictions could be shown as suggestions.

Confidence-based selective prediction. Not all predictions are equally reliable. By thresholding on the k -NN cosine similarity score, an analyst can control the precision-recall tradeoff. At a threshold of 0.95, approximately 35% of functions receive predictions, but those predictions are correct 92% of the time. This selective prediction mode is more useful in practice than predicting all functions with moderate accuracy, because wrong names are worse than no names because they actively mislead analysts.

Updatable knowledge base. The k -NN datastore is an explicit, updatable knowledge base. When an analyst manually identifies a function, its embedding can be added to the datastore immediately, improving predictions for similar functions *without retraining the model*. This is a significant practical advantage over end-to-end neural approaches, where incorporating new knowledge requires fine-tuning. Over time, the datastore can be specialized to the analyst’s domain (e.g., adding IoT firmware functions, malware family-specific patterns).

7.5 Comparison with LLM Approaches

The trend in binary analysis is toward ever-larger language models: SYMGEN [5] fine-tunes CodeLlama-34B, ProRec [7] feeds symbol-rich context to black-box foundation models, and Beyond Classification [10] uses domain-adapted LLMs. These approaches achieve strong in-distribution performance but face practical challenges:

- **Cost:** Fine-tuning a 34B-parameter model requires multiple high-memory GPUs and many hours per epoch. FUNCr trains in under 4 GPU-hours on a single A100-80GB.
- **Inference speed:** LLM inference is bottlenecked by autoregressive generation. FUNCr’s k -NN mode processes 13,301 functions in 72 seconds, an order of magnitude faster per function.
- **Cross-project generalization:** The released SYMGEN checkpoint, when reproduced on our four-package cross-project test set, achieves 0.450 F1 while FUNCr achieves 0.650. Larger models do not automatically generalize better, especially when retrieval suffices for the target domain.
- **Pre-training contamination:** CodeLlama’s 2T-token pre-training corpus almost certainly includes the open-source projects we evaluate against (upstream nginx, GNU coreutils). FUNCr’s from-scratch training on BAP-IR has no such implicit exposure, making small-model results cleaner scientifically.
- **Updatability:** Adding new function patterns to an LLM requires fine-tuning; adding them to a k -NN datastore is instant.

We do not argue that FUNCr is universally superior to LLM approaches; LLMs may excel at compositional naming and have access to broader world knowledge. However, for the specific task of cross-project function name recovery, a well-designed encoder with retrieval-based inference provides a strong, efficient alternative.

7.6 Structure-Aware vs. Source-Code LLM Paradigms

The recent literature on binary function name recovery has bifurcated into two complementary paradigms. Large-scale *source-code LLM* approaches [5, 7, 10] leverage textual and stylistic priors learned from source-code pretraining corpora: they map decompiled pseudo-code to names by exploiting patterns their billion-parameter backbones absorbed from GitHub-scale corpora. In contrast, *structure-aware* approaches—including our work as well as BLens [4], XFL [6], SymLM [3], TYGR [17], CALLEE [16], and CFG2VEC [9]—directly encode binary-intrinsic features (control-flow graph structure, call graphs, instruction-level semantics, inter-procedural signatures) using small-to-medium models trained on binary representations rather than source text. Both paradigms have produced strong published results at top-tier venues in the 2022–2025 window; they are not competing claims for a single “correct” approach but complementary views of the problem.

FUNCR is deliberately in the structure-aware camp. Beyond the efficiency and deployability arguments already discussed, this design choice has a more subtle implication: a model that learns a CFG structure $\rightarrow$ name mapping is *invariant* to shifts in how humans (or language models) verbalize that structure in source code. The control-flow graph of a hash-table insertion function is the same whether the source labels it `hash_insert`, `ht_put`, or `insertIntoTable`; only the label distribution shifts. Source-code LLM approaches that learn from textual patterns are, by construction, sensitive to the text distribution—if naming conventions drift, the model’s learned text-to-name mapping drifts with them.

This robustness-to-text-distribution-shift has an increasingly salient forward-looking dimension. AI-assisted coding tools (GitHub Copilot, Cursor, and similar LLM-driven assistants) have become mainstream in new software development, and there is growing informal evidence that their suggestions homogenize naming styles across codebases by aggregating over human conventions observed during pretraining. Over time, the textual distribution of function names in real-world code will likely become a mixture of human idioms and LLM-generated conventions, shifting away from the distribution that today’s source-code LLMs were pretrained on. Structure-aware models that tie predictions to computational invariants (how the function behaves) rather than to surface lexical patterns (how it is written) should, in principle, degrade more gracefully under such shifts. We emphasize that this is a hypothesis, not a demonstrated result—empirical validation would require longitudinal studies of function naming in AI-assisted codebases, which we leave to future work. But it motivates continued investment in structure-aware approaches alongside the current LLM-based momentum in the field.

8 LIMITATIONS

- (1) **Pure recognizer:** The model cannot compose novel function names. All correct predictions are names seen in training. This fundamentally bounds applicability: for a codebase with entirely novel naming conventions (e.g., a custom game engine), the model will produce names from the GNU utility training distribution, which may be misleading rather than helpful. The k -NN datastore partially

mitigates this by making the knowledge base explicit and updatable, but the underlying encoder representations still reflect training data biases.

- (2) **O0/O2 gap:** O0 binaries remain significantly harder (41.5% vs. 69.8% EM). O0 code produces many small, structurally similar functions that the model cannot distinguish.
- (3) **Version sensitivity:** Different versions of the same program produce near-zero accuracy (http: 0.6% EM despite being in training), because function bodies change across versions while names may stay the same or change unpredictably.
- (4) **BAP dependency:** Our pipeline requires BAP 2.5.0 for IR lifting, which is slower than disassembly-based approaches (e.g., IDA Pro or Ghidra). Processing a single binary takes 30–120 seconds depending on size. A production deployment would benefit from a faster lifter or direct disassembly-based tokenization.
- (5) **Approximate comparison:** Different datasets and evaluation protocols prevent exact comparison with prior work. We mitigate this by following SYMGEN’s deduplication methodology and reporting cross-project metrics, but differences in package selection, compiler versions, and optimization levels introduce confounds.
- (6) **Platform and architecture scope:** We evaluate only x86-64 ELF binaries compiled with GCC on Linux. Windows PE binaries, ARM/MIPS architectures, and other compilers (Clang, MSVC) are not tested. The instruction-type tokenization is architecture-specific and would require reimplementation for other ISAs, though the high-level approach (semantic type abstraction) is transferable.
- (7) **No obfuscation handling:** We assume binaries are not deliberately obfuscated. Control flow flattening, opaque predicates, and instruction substitution would defeat both the CFG-based graph encoder and the instruction-type tokenization. Evaluating robustness to obfuscation is an important direction for future work.

9 FUTURE WORK

Several directions could extend FUNCR’s capabilities:

Cross-architecture support. Extending FUNCR to ARM, MIPS, and RISC-V architectures would require adapting the instruction-type tokenization to each ISA. The high-level approach of classifying instructions into semantic types (memory operations, arithmetic, control flow) is architecture-agnostic, but the specific token definitions would differ. A unified tokenization across architectures could enable cross-architecture transfer learning, where knowledge from well-studied x86-64 binaries improves predictions on less common architectures.

Compositional naming. The recognizer limitation is the fundamental ceiling on FUNCR’s accuracy. Breaking through this ceiling requires compositional generation: predicting names for functions that were never seen in training by composing sub-tokens based on semantic understanding. This is a fundamentally harder problem because it requires the model to understand what a function *does* (e.g., “this function sorts a linked list”) and translate that understanding into a name. Hybrid approaches that combine retrieval

(for known functions) with generation (for unknown functions) are a promising direction, using the k -NN confidence score to decide when to trust generation.

Incremental learning. In a practical reverse engineering workflow, the analyst progressively identifies functions. Each identified function could be added to the k -NN datastore, potentially improving predictions for related functions. An active learning framework that suggests which functions the analyst should examine next (those with the highest expected information gain for the datastore) could further accelerate the workflow.

Obfuscation robustness. Evaluating and improving robustness to binary obfuscation (control flow flattening, opaque predicates, dead code insertion) is important for malware analysis applications. The contrastive pre-training objective could be extended to include obfuscated variants as positive pairs, encouraging the encoder to learn obfuscation-invariant representations.

10 CONCLUSION

We presented FUNCr, a graph neural network system for binary function name recovery that achieves 0.779 F1 on a held-out test set (13,301 functions) and 0.650 F1 across four completely unseen packages (10,467 functions). On the same cross-project test set, we reproduce the released SYMGEN checkpoint (34B-parameter CodeLlama) and measure 0.450 F1, establishing a +0.20 absolute improvement for FUNCr with a model roughly 1000× smaller. Our key contributions are:

- (1) A **multi-context gated fusion mechanism** with conditional bypass that sequentially incorporates external library calls, callee signatures, and caller signatures. Our ablation study reveals the *ext-call paradox*: external calls alone improve test performance but degrade cross-project generalization by 4.3 percentage points. Inter-procedural callee/caller context resolves this paradox (+12.8pp cross-project EM), demonstrating that naive feature addition can harm generalization and explaining why cascaded fusion with conditional bypass is necessary.
- (2) The finding that **k -NN retrieval outperforms autoregressive decoding** on cross-project generalization with zero retraining. For our encoder architecture, retrieval over embeddings is more effective than generation: the autoregressive decoder’s compositional ability is not just unused but actively harmful (43.8% hallucinated predictions on the ablation subset). The encoder representation, not the decoder, is the key component.
- (3) A **comprehensive ablation study** quantifying the contribution of each component across four model variants, from a basic 4.4M-parameter GAT baseline to the 25M-parameter pretrained ablation system, scaled up to a final 32M-parameter model trained on 300K functions. We demonstrate that multi-context fusion provides +0.175 test F1, scaling resolves cross-package contamination, and pre-training enables partial embedding transfer across vocabulary changes.

Our results demonstrate that binary function name recovery can be effectively solved with a 32M-parameter encoder and simple

nearest-neighbor retrieval, achieving superior performance to systems based on billion-parameter language models at a fraction of the computational cost. The k -NN datastore additionally provides an updatable, inspectable knowledge base that can grow with analyst expertise, a practical advantage for deployment in real-world reverse engineering workflows.

REFERENCES

- [1] J. He, P. Ivanov, P. Tsankov, V. Raychev, and M. Vechev, “Debin: Predicting debug information in stripped binaries,” in *Proc. ACM CCS*, 2018, pp. 1667–1680.
- [2] Y. David, U. Alon, and E. Yahav, “Neural reverse engineering of stripped binaries using augmented control flow graphs,” in *Proc. ACM OOPSLA*, 2020.
- [3] X. Jin, K. Pei, J. Y. Won, and Z. Lin, “SymLM: Predicting function names in stripped binaries via context-sensitive execution-aware code embeddings,” in *Proc. ACM CCS*, 2022, pp. 1631–1645.
- [4] T. Benoit, Y. Wang, M. Dannehl, and J. Kinder, “BLens: Contrastive captioning of binary functions using ensemble embedding,” in *Proc. USENIX Security*, 2025.
- [5] Y. Xu, K. Xu, and D. Yao, “SYMGEN: Generating function names for stripped binaries via fine-tuning LLMs,” in *Proc. NDSS*, 2025.
- [6] J. Patrick-Evans, M. Dannehl, and J. Kinder, “XFL: Naming functions in binaries with extreme multi-label learning,” in *Proc. IEEE S&P*, 2023.
- [7] Z. Su, X. Xu, Z. Huang, K. Zhang, and X. Zhang, “Source code foundation models are transferable binary analysis knowledge bases,” in *Proc. NeurIPS*, 2024.
- [8] X. Xu, S. Feng, Y. Ye, H. Shen, Z. Su, S. Cheng, G. Tao, Q. Shi, Z. Zhang, and X. Zhang, “Enhancing function name prediction using votes-based name tokenization and multi-task learning,” in *Proc. ACM FSE (PACMSE)*, 2024.
- [9] S.-Y. Yu, Y. G. Achamyeh, C. Wang, A. Kocheturov, P. Eisen, and M. A. Al Faruque, “CFG2VEC: Hierarchical graph neural network for cross-architectural software reverse engineering,” in *Proc. IEEE/ACM ICSE SEIP*, 2023.
- [10] L. Jiang, X. Jin, and Z. Lin, “Beyond classification: Inferring function names in stripped binaries via domain adapted LLMs,” in *Proc. NDSS*, 2025.
- [11] X. Xu, Z. Zhang, Z. Su, Z. Huang, S. Feng, Y. Ye, N. Jiang, D. Xie, S. Cheng, L. Tan, and X. Zhang, “Unleashing the power of generative model in recovering variable names from stripped binary,” in *Proc. NDSS*, 2025.
- [12] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, “Neural network-based graph embedding for cross-platform binary code similarity detection,” in *Proc. ACM CCS*, 2017, pp. 363–376.
- [13] S. H. Ding, B. C. Fung, and P. Charland, “Asm2Vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization,” in *Proc. IEEE S&P*, 2019, pp. 472–489.
- [14] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, “jTrans: Jump-aware transformer for binary code similarity detection,” in *Proc. ACM ISSTA*, 2022, pp. 1–13.
- [15] K. Pei, Z. Xuan, J. Yang, S. Jana, and B. Ray, “Trex: Learning execution semantics from micro-traces for binary similarity,” in *IEEE Trans. Software Engineering*, 2022.
- [16] Q. Zhu, Z. Feng, L. Zhang, and Y. Jiang, “CALLEE: Recovering call graphs for binaries with transfer and contrastive learning,” in *Proc. IEEE S&P*, 2023.
- [17] C. Zhu, Z. Li, A. Xue, A. P. Bajaj, W. Gibbs, Y. Liu, R. Alur, T. Bao, H. Dai, A. Doupe, M. Naik, Y. Shoshitaishvili, R. Wang, and A. Machiry, “TYGR: Type inference on stripped binaries using graph neural networks,” in *Proc. USENIX Security*, 2024.
- [18] H. He, X. Lin, Z. Weng, R. Zhao, S. Gan, L. Chen, Y. Ji, J. Wang, and Z. Xue, “Code is not natural language: Unlock the power of semantics-oriented graph representation for binary code similarity detection,” in *Proc. USENIX Security*, 2024.
- [19] D. Brumley, I. Jager, T. Avgerinos, and E. J. Schwartz, “BAP: A binary analysis platform,” in *Proc. CAV*, 2011, pp. 463–469.
- [20] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis, “Generalization through memorization: Nearest neighbor language models,” in *Proc. ICLR*, 2020.
- [21] U. Khandelwal, A. Fan, D. Jurafsky, L. Zettlemoyer, and M. Lewis, “Nearest neighbor machine translation,” in *Proc. ICLR*, 2021.
- [22] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, D. de Las Casas, A. Guy, J. Menick, R. Ring, T. Hennigan, S. Huang, L. Maggiore, C. Jones, A. Cassirer, A. Brock, M. Paganini, G. Irving, O. Vinyals, S. Osindero, K. Simonyan, J. W. Rae, E. Elsen, and L. Sifre, “Improving language models by retrieving from trillions of tokens,” in *Proc. ICML*, 2022.
- [23] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, “Scheduled sampling for sequence prediction with recurrent neural networks,” in *Proc. NeurIPS*, 2015.
- [24] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. ICLR*, 2018.